The Journal of Open Source Software

CtrlAer: A MicroPython package and embedded domain specific language for real-time control of piezoelectric atomisers

S. Hessam M. Mehr [1]

1 Leverhulme Early Career Fellow, University of Glasgow, United Kingdom

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 23 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

As an emerging medium for investigating chemical reactivity, aerosols offer access to massive experimental throughput, are amenable to direct analysis via mass spectrometry, and promise reactivity patterns distinct from chemical reactions carried out within bulk solution. The adoption of aerosols by chemistry researchers requires a robust and cost effective method to generate them in a reproducible fashion, ideally via methodology amenable to automation. In reality, current aerosol generation options are optimised for studying the their physical properties or simulating their production in the environment, including their role as pollutants. The evolution of aerosol particles is also inherently more rapid than laboratory setups using , requiring especial attention to maintaining accurate experimental timings. Aerosol microdroplets are prone to rapid equilibration (via evaporation or ripening) or precipitation, so manual release or manipulation is often not practical. `CtrlAer` is a library targeting the Raspberry Pi RP2040 and RP2350 microcontrollers that enables programmable control of aerosol sources via a domain-specific syntax embedded within MicroPython.

Statement of need

`CtrlAer`'s primary aim is to lower the barrier to entry for scientists and engineers looking to undertake experiments involving aerosols. Commercial vibrating mesh atomisers (VMAs) provide a robust and inexpensive hardware option for producing bespoke aerosols but their accessibility is currently limited by the lack of a software option for precise control, including synchronisation of multiple atomisers with auxiliary lab devices such as valves and relays. It has already been deployed in a recent publication (Zhang & Mehr, 2024).

CtrlAer uses the microcontroller's programmable input/output (PIO) subsystem for signal generation, freeing up the processor to handle other tasks, such as event processing, or data logging. The RP2040 and RP2350 have two and three PIO units respectively, with each PIO unit consisting of four state machines using a shared clock signal. The PIO's built-in first-in-first-out (FIFO) allows gap-less signal generation, with as many as 16 general-purpose IO (GPIO) pins controllable by a single state machine, Figure 1A.

Two types of signals are currently supported: square waves for driving VMAs and DC high/low signals for controlling other devices. Square wave frequency can be controlled on the PIO level, so two (RP2040) or three (RP2350) independent frequencies are possible. Frequency setup is possible both during setup via the `CtrlAer` constructor as well as in real-time by calling the `set_freq` method. The phase, i.e. `0101` vs `1010` of a square wave can further be specified on a per-pin basis.

One of `CtrlAer`'s principal aims is to offer an abstraction over the low-level PIO mechanics by providing domain-specific constructs for generation of different signal types. Each signal is

represented as a Python generator, allowing instructions to be produced dynamically in response to measurements and without requiring the entire sequence to fit within the microcontroller's limited memory. The tuples yielded by each generator are of the form `<COMMAND> <DURATION>`, where `<DURATION>` is a time period specified by default in milliseconds. At runtime, the `<COMMAND>` along with `ticks` (i.e. the number of state machine cycles calculated based on `<DURATION>` and PIO frequency) are placed on the PIO state machine's FIFO sequentially as 32-bit values. `<COMMAND>` can be any of the following, Figure 1B.

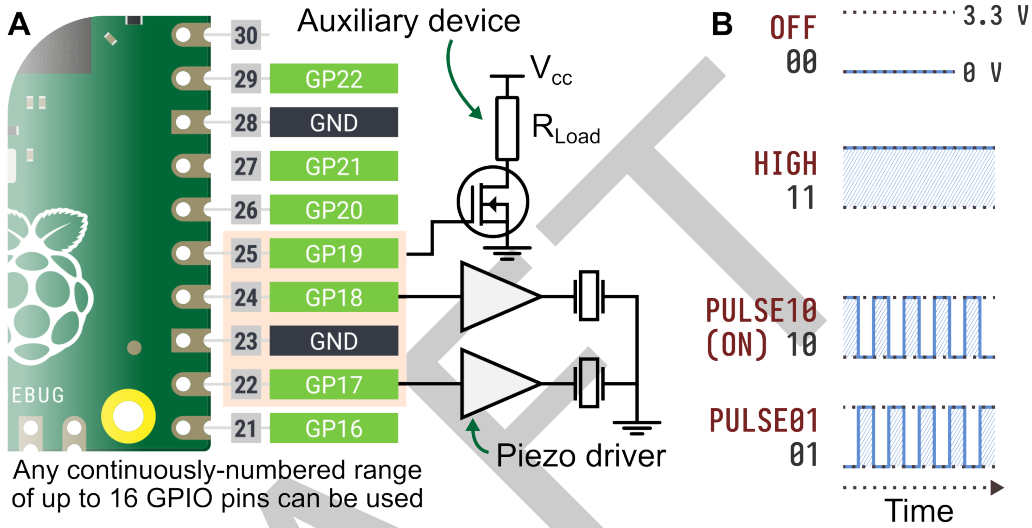

Figure 1: CtrlAer's operation at a glance. (A) Schematic description of typical experimental setup using a Raspberry Pi Pico development board running CtrlAer along with an auxiliary controlled device — represented by its load resistance — and two VMAs. (B) Signal types available within CtrlAer for each channel.

- **`OFF`:** Continuously low (0 V) signal level for duration of command. Binary representation: `00`.
- **`PULSE10`:** Square wave beginning with high (3.3 V) signal level. Binary representation `10`.
- **`PULSE01`:** Square wave beginning with low (0 V) signal level. Binary representation: `01`.
- **`HIGH`:** Constant high (3.3 V) signal level. Binary representation: `11`.
- **`ON`:** Alias for `PULSE10`. Binary representation: `10`.

Internally, the state machine alternates between the `COMMAND`'s two bits, performing a single-bit right shift from the output shift register (OSR) to the pins, repeating this `ticks` times.

The above per-pin constructs can be combined for synchronised control of any continuous range of GPIO pins through the use of another CtrlAer construct dubbed `mux`. Specifically, the `mux` function combines N generators, each corresponding to 1 GPIO pin and returns a generator capable of controlling a continuous range of N pins. Internally, `mux` interlaces the two state bits of each supplied command and calculates the next largest time window where all pin signals are valid, Figure 2.

DRAFT

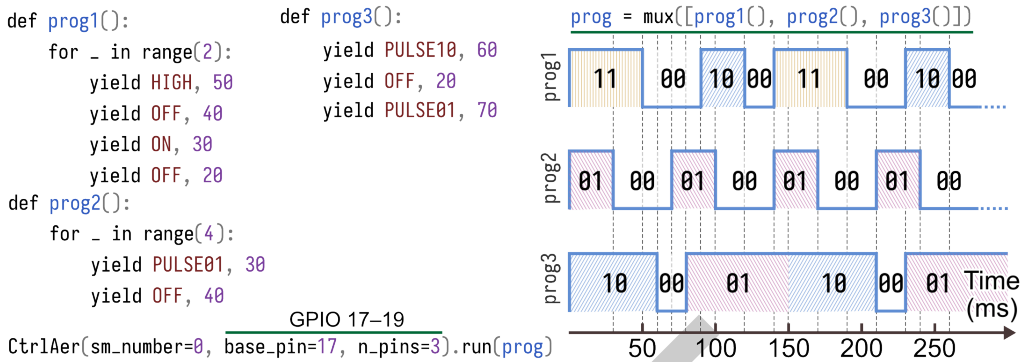

Figure 2: Typical CtrlAer program for the three-channel setup shown in Figure 1. The diagram on the right shows the multiplexed time segments generated by the `mux` function.

Some of CtrlAer's functionality is dependent on a sister library, `rp2040hw`, which exposes the RP2040/RP2350's hardware registers as Python data structures for low-level device control. `CtrlAer` specifically relies on access to PIO registers to manipulate a PIO's frequency without having to stop and restart it.

In our testing, CtrlAer has been capable of gap-free signal generation with segments as small as 1 ms, though intervals as short as a single square wave period can be produced transiently. Because Python generators are used to create successive signal segments, complex logic and/or external input can be used for dynamic generation, so long as the required computation time is on average shorter than the duration of the corresponding signal segments.

Acknowledgements

This work was supported by the Leverhulme Trust (grant ECF-2021-298), Royal Society of Chemistry (grant E22-7895308996), and the University of Glasgow Lord Kelvin Adam Smith Leadership Fellowship.

References

Zhang, L., & Mehr, S. H. M. (2024). In situ synthesis within micron-sized hydrogel reactors created via programmable aerosol chemistry. *Digital Discovery*. https://doi.org/10.1039/D4DD00139G

DRAFT